Esqueletos: qué tocar según el pedido

> Archivo personal (no se sube). Comandos: `COMANDOS.local.md`. Lo básico de los lenguajes: `LENGUAJES.local.md`.

El camino de un pedido

```
clic (.html) → método (.ts) → service del front (http)
  → RUTA (portero: 401 / 403) → VALIDATOR (400)
  → CONTROLLER (req → service → res.json)
  → SERVICE (reglas: 404 / 403 / 409 + SQL) → base
  → JSON → mapeo → signal → la pantalla se actualiza sola
```

¿Qué tipo de pedido es? (y qué hay parecido en el código)

El pedido dice...	Tipo	Tocás, en orden	Parecido que ya existe (`Ctrl + Shift + F`)
mostrar / filtrar / buscar en lo que ya se ve	solo front	`.ts` (signal + computed) → `.html`	buscador de **Reservas del propietario**: `busqueda` + `filtradas` en `pages/propietario/reservas/reservas.ts`
agregar un texto a algo que existe	campo nuevo	schema → validator → service → dto/model/mapeo → form → pantalla	`color` del vehículo · `motivo_reasignacion` de la reserva
agregar un sí/no a algo que existe	campo nuevo	igual que arriba (checkbox)	`cubierto` del estacionamiento · `predeterminado` del vehículo
un sí/no + poder filtrar por eso	campo + filtro	lo de arriba + `validarBusqueda` + `buscar` + chip	`cubierto` → `soloCubiertos` (Explorar → Filtros)
que el conductor **haga** algo sobre algo suyo	acción	schema? → service → controller → ruta → front service → botón	**cancelar reserva** (`/reservas/:id/cancelar`)
que el propietario **haga** algo sobre una reserva	acción	ídem	**confirmar** / **marcarAusente** (`reserva-ciclo.service.js`)
ver un resumen / cuántos / por cada	consulta	service (SELECT + GROUP BY) → controller → ruta (GET) → front	`services/estadistica/estadistica-reservas.js` · `GET /:id/reservas`
que **no** se pueda X si Y	regla	solo el service: consulta + `if` + `throw` antes de guardar	las reglas de `cancelar` y `crear` en `reserva.service.js`
crear / borrar algo que pertenece a otra cosa	tabla nueva	CREATE TABLE + POST / DELETE	**planes de abono**: `POST` y `DELETE /:id/planes-abono` · tabla `horario`
algo del admin	acción admin	`admin.service/controller/routes` (ya tienen portero)	`aprobar` estacionamiento · `ocultar` reseña
un botón en una tarjeta / fila	front	`output` + `emit` en el hijo, `subscribe` en el padre	"Eliminar" en `tarjeta-vehiculo` · "Marcar ausente" en `seccion-reservas`

¿En qué archivo va?

1. **Escribí la URL** que querés: `PATCH /reservas/:id/x`.
2. **El primer tramo dice el archivo de rutas** (`routes/index.js`): `/reservas` → `reserva.routes.js` · `/estacionamientos` → `estacionamiento/estacionamiento.routes.js` (salvo `/:id/cocheras` → `estacionamiento-cochera.routes.js`) · `/vehiculos` → `vehiculo.routes.js` · `/admin` → `admin.routes.js`.
3. **El controller es el que ese archivo de rutas ya importa** (mirá los `import` de arriba).
4. **El service es el que ese controller ya importa.** Si hay varios, el de la función parecida que estás copiando.
5. **La URL** se arma como las de al lado: `/<cosa en plural>/:id/<acción>` (ej. `/estacionamientos/:id/aprobar`).

Dónde está lo más tocado:

Busco	Archivo
crear / listar / cancelar reserva	`services/reserva/reserva.service.js`
acciones del propietario (confirmar, ausente)	`services/reserva/reserva-ciclo.service.js`

Busco	Archivo
el SELECT de todas las reservas	`services/reserva/reserva-detalle.js` (`selectDetalle`)
alta / edición / `COLUMNAS` de estacionamiento	`services/estacionamiento/estacionamiento.service.js`
filtros del Explorar	`services/estacionamiento/estacionamiento-busqueda.service.js`
vehículos	`services/vehiculo.service.js`

Si no sabés dónde está algo: `Ctrl + Shift + F` con el nombre de una función o columna parecida.

Esqueletos para copiar

Acción completa del back

```
// SERVICE
export async function marcarX(idCosa, idUsuario) {
  const { rows } = await query(
    'SELECT id_dueno, estado FROM tabla WHERE id_tabla = $1', [idCosa]);
  if (!rows[0]) throw ApiError.notFound('No existe');
  if (rows[0].id_dueno !== idUsuario) throw ApiError.forbidden('No es tuyo');
  await query('UPDATE tabla SET x = TRUE WHERE id_tabla = $1', [idCosa]);
}

// CONTROLLER
export const marcarX = asyncHandler(async (req, res) => {
  await servicio.marcarX(req.params.id, req.usuario.id);
  res.json({ ok: true });
});

// RUTA
router.patch('/:id/x', ...soloConductor, controller.marcarX);
```

Si la acción es del propietario sobre una reserva, en `reserva-ciclo.service.js` ya está `bloquearReservaDelPropietario(client, idReserva, idPropietario)`: hace el 404 y el 403 en una línea (la usa `confirmar`).

Consulta (GET) completa del back

```
// SERVICE
export async function contarX(idCosa) {
  const { rows } = await query(
    `SELECT campo, COUNT(*)::int AS cantidad FROM tabla
      WHERE id_otro = $1 GROUP BY campo`, [idCosa]);
  return rows;
}
// CONTROLLER
export const contarX = asyncHandler(async (req, res) => {
  const filas = await servicio.contarX(req.params.id);
  res.json({ filas });
});
// RUTA
router.get('/:id/conteo', ...soloPropietario, controller.contarX);
```

Para chequear que el estacionamiento sea del propietario: ya existe `asegurarPropiedad(idEstacionamiento, idPropietario)` (la usa `listarPorEstacionamiento`).

Regla (adentro de la función que crea, antes del INSERT)

```
const { rows: cuenta } = await client.query(
  'SELECT COUNT(*)::int AS n FROM tabla WHERE id_usuario = $1', [id]);
if (cuenta[0].n >= 3) throw ApiError.conflict('Máximo 3');
```

Tabla nueva

```
CREATE TABLE IF NOT EXISTS tabla_nueva (
  id_usuario UUID NOT NULL
    REFERENCES usuario (id_usuario) ON DELETE CASCADE,
  id_estacionamiento UUID NOT NULL
    REFERENCES estacionamiento (id_estacionamiento) ON DELETE CASCADE,
  PRIMARY KEY (id_usuario, id_estacionamiento)
);
```

```
await query(`INSERT INTO tabla_nueva (id_usuario, id_estacionamiento)
             VALUES ($1, $2) ON CONFLICT DO NOTHING`, [idU, idE]);
await query(`DELETE FROM tabla_nueva
              WHERE id_usuario = $1 AND id_estacionamiento = $2`, [idU, idE]);
```

Test

```
describe('lo que pruebo', () => {
  test('hace X', async () => {
    const { propietario, conductor, estacionamiento, vehiculo } =
      await escenario();                        // en reserva.test.mjs
    const r = await api('PATCH', `/reservas/${id}/x`,
      { token: conductor.token, body: { campo: 'valor' } });
    console.log(r.estado, r.datos);             // mirar qué volvió
    assert.equal(r.estado, 200);
  });
});
```

- Ayudas: `crearUsuario()` · `crearAdmin()` · `crearVehiculo(token)` · `crearEstacionamiento(token)` · `franja('08:00', '09:00')`.
- Tests parecidos para copiar: `'el conductor cancela la suya, una sola vez'` (reserva) · `'edita marca y modelo'` (vehículo) · `'cambia datos y rearma la direccion'` (estacionamiento).
- Códigos: sin token 401 · otro rol 403 · datos mal 400 · no existe 404 · de otro 403/404 · estado que no corresponde 409.
- El `console.log` no es parte del test: es para ver qué volvió.

Botón en el front

```
// service del front (adentro de la clase)
marcarX(id: Id) {
  return this.http.patch(`${this.ruta}/${id}/x`, {});
}
// pantalla .ts
protected marcarX(id: Id): void {
  this.servicio.marcarX(id).subscribe(() => this.recurso.reload());
}
```

```
<button type="button" (click)="marcarX(item.id)">Marcar</button>
```

Si el botón está en un **hijo** (tarjeta, sección): `output` + `.emit` en el hijo, `(evento)="metodo($event)"` + `subscribe` en el padre. Igual que "Eliminar" en `tarjeta-vehiculo` → `pedirEliminacion` en `vehiculos.ts`.

Campo de texto en el front (buscador, comentario, nota...)

```
// pantalla .ts: un signal que guarda lo que escriben
protected readonly texto = signal('');
```

```
<label class="campo">
  <span class="label-campo">Comentario</span>
  <input
    placeholder="Escribí algo"
    [value]="texto()"
    (input)="texto.set($any($event.target).value)"
  />
</label>
```

- `[value]="texto()"` → muestra lo que tiene el signal.

- `(input)="..."` → en cada tecla guarda lo escrito en el signal.
- `$any($event.target).value` = "el texto que hay en el input". **Copiarlo tal cual.** (`$event.target` es el input; `$any` es para que TypeScript no se queje.)
- Después lo usás con `this.texto()`: para filtrar en un `computed`, o para mandarlo al back: `this.servicio.crear({ ...payload, comentario: this.texto() })`.
- En el código ya hay uno igual: el buscador de **Reservas del propietario** (`filtros-reservas.html`, usa `busqueda.set(...)`).

Variantes:

```
<!-- select: (change) en vez de (input) -->
<select (change)="estado.set($any($event.target).value)">
  <option value="">Todos</option>
  <option value="PENDIENTE">Pendientes</option>
</select>

<!-- checkbox suelto: .checked en vez de .value -->
<input type="checkbox"
  (change)="soloMios.set($any($event.target).checked)" />

<!-- número: pasarlo con Number(...) -->
<input type="number"
  (input)="cantidad.set(Number($any($event.target).value))" />
```

Si el campo está en un **formulario reactivo** (alta/edición), no va así: va con `formControlName` (ver `LENGUAJES.local.md` y el `formulario-vehiculo`).

Errores → dónde mirar

Veo	Causa
500	bug del back: `[error]` en la terminal del back / del test
`x is not defined`	variable mal escrita (`idReserva` ≠ `id_reserva`)
`Cannot read properties of undefined (reading 'X')`	lo que está antes de `.X` vino vacío (`row` en vez de `rows`)
`column "x" does not exist`	typo, o el schema no se aplicó
`syntax error at or near "X"`	SQL mal justo antes de X (falta tipo, coma, alias)
`bind message supplies N...`	no coinciden los `$` con el array
404 en ruta nueva	la URL no coincide (`_` ≠ `-`) o falta la ruta
400	falta el campo en el validator (o en `CAMPOS_EDITABLES`)
campo `undefined` en la respuesta	falta en el SELECT / `COLUMNAS` / mapeo
`ng build`: `Cannot find name 'x'`	método pegado **afuera** de la clase
`Property 'x' does not exist on type`	lo usás en el html y no está en el .ts
`Cannot find control with name 'x'`	falta en el form del .ts
botón que no hace nada	F12 → Network: sin pedido = front; en rojo = back (mirar Status y Response)
no aparece algo en pantalla	el `@if` da false: imprimí `{{ algo }}`

Orden de trabajo

1. Leé el pedido y ubicalo en la tabla de tipos.
2. Buscá lo parecido (`Ctrl + Shift + F`) y abrí esos archivos.
3. Base → back → **probá** (test o Network).
4. Front → `npx ng build` → navegador.
5. `git add` + `git commit`.
6. Si se acaba el tiempo: que el back funcione y se vea algo.